Hafta 7 — State Management: Cookie ve Session

> **Ders:** Web Programlama · **Hafta:** 7 / 13 ders haftası · **Süre:** 3 × 50 dakika **Ön koşul:** Hafta 1 (HTTP durumsuzdur), Hafta 2 (middleware sırası), Hafta 4 (`TempData` , istemci verisine güvenmemek), Hafta 5 (anti-forgery cookie'si) **Bu dosya sınav kapsamındadır.** Daha derin konular `H07-detay.md` dosyasındadır ve **sınav kapsamı dışıdır.**

🎯 Bu Haftanın Hedefleri

Bu haftanın sonunda:

1. HTTP durumsuzken bir uygulamanın kullanıcıyı nasıl "hatırladığını" ve durumun **nerede** tutulabileceğini **anlatabileceksiniz**.
2. `Set-Cookie` ve `Cookie` başlıklarıyla bir cookie'nin yolculuğunu **izleyebileceksiniz**.
3. `Response.Cookies.Append` ile cookie **yazabilecek**, `Request.Cookies["…"]` ile **okuyabilecek** ve okuduğunuz değere neden güvenmemeniz gerektiğini açıklayabileceksiniz.
4. `CookieOptions` ile `Expires` , `HttpOnly` , `Secure` ve `SameSite` seçeneklerinin ne yaptığını **ayırt edebileceksiniz**.
5. Session'ı `AddSession` ve `UseSession` ile **kurabilecek**, eksik kurulumun üç farklı hatasını tanıyabileceksiniz.
6. Session'a `SetString` / `SetInt32` ile değer, **JSON'a çevirerek** nesne yazabileceksiniz.
7. Kullanıcının rolünü cookie'de tutmanın neden açık olduğunu **gösterebileceksiniz**.
8. Bir veriyi cookie'de mi, session'da mı, `TempData` 'da mı, veritabanında mı tutacağınıza **karar verebileceksiniz**.

🧩 **Haftanın ilkesi — Durum bir yerde tutulmak zorundadır; soru "nerede" ve "kime ait" sorusudur**

> HTTP hiçbir şeyi hatırlamaz. Hatırlanması gereken her şey ya kullanıcının tarayıcısında ya da sizin sunucunuzda durur. Tarayıcıda duran veri kullanıcınındır: okur, değiştirir, siler. Sunucuda duran veri sizindir.

---

1. Durum Nerede Tutulur?

Durum (state), bir isteğin bittiği yerde kaybolmaması gereken bilgidir: sepetteki kitaplar, kullanıcının giriş yapmış olması, seçtiği tema. **State management** bu bilgiyi istekler arasında taşımanın yollarıdır.

1.1. Hafta 1'e dönüş: sunucu hatırlamaz

Hafta 1'de HTTP'nin **durumsuz (stateless)** olduğunu gördük: her istek bağımsızdır, sunucu bir önceki isteği kendiliğinden hatırlamaz. Bir kitabı sepete eklediğiniz istek bittiğinde sunucuda o isteğe ait hiçbir şey kalmaz. Bir sonraki sayfayı açtığınızda sunucu karşısındakinin aynı kişi olduğunu **bilmez**.

Bu bir eksiklik değil, bir tasarım kararıdır: sunucu kimseyi hatırlamak zorunda olmadığı için binlerce isteğe aynı anda cevap verebilir. Ama uygulamanın hatırlaması gerekiyorsa bilgi **bir yere** konmalıdır.

1.2. Beş yer

Yer	Veri nerede durur	Kim değiştirebilir	Ne kadar yaşar	Bu derste
Query string	Adreste: `/Kitap/Ara?kelime=suc`	Kullanıcı (adres çubuğu)	Bağlantı kadar	Hafta 2, 4
Form alanı (gizli alan dahil)	Sayfanın HTML'inde, sonra istek gövdesinde	Kullanıcı (F12)	Tek gönderim	Hafta 4
Cookie	**Tarayıcıda**	**Kullanıcı**	Siz ne derseniz (dakika – yıl)	**Bu hafta**
Session	**Sunucuda**; tarayıcıda yalnızca anahtarı	Yalnızca sunucu	Hareketsizlikte ~20 dk	**Bu hafta**
Veritabanı	Sunucuda, diskte	Yalnızca sunucu	Siz silene kadar	Hafta 8

Tablonun üçüncü sütunu haftanın ilkesidir. İlk üç satırdaki veri **kullanıcının elinde** durur; Hafta 4'ün ilkesi — *istemciden gelen hiçbir veri güvenilir değildir* — cookie için de aynen geçerlidir.

1.3. Aslında iki haftadır cookie kullanıyorsunuz

Projede `/Durum` sayfasını açın. Tarayıcının o istekle getirdiği cookie'ler tablo hâlinde listelenir. Bir kitap ekleyip yönlendirildikten sonra şunları görürsünüz:

Cookie	Kim yazdı	Hafta
`.AspNetCore.Mvc.CookieTempDataProvider`	`TempData` — "kaydedildi" mesajını yönlendirmenin öbür tarafına taşıyan	4
`.AspNetCore.Antiforgery.…`	Form Tag Helper — CSRF token çiftinin cookie yarısı	5

`TempData` cookie'si okunduktan sonra silinir: ölçülen yanıtta `.AspNetCore.Mvc.CookieTempDataProvider=; expires=Thu, 01 Jan 1970 00:00:00 GMT` satırı gelir. Süresi geçmişte olan cookie tarayıcı tarafından hemen atılır — bir cookie'yi silmenin yolu budur (bölüm 2.5). Bu hafta bu mekanizmayı **kendimiz** kullanacağız.

---

2. Cookie

Cookie, sunucunun tarayıcıya bıraktığı ve tarayıcının aynı siteye yaptığı **her istekte kendiliğinden geri getirdiği** küçük bir `ad=değer` çiftidir.

2.1. Nasıl çalışır: iki başlık

Cookie'nin bütün mekanizması iki HTTP başlığıdır:

```
1) Sunucu → tarayıcı (yanıt başlığı)
HTTP/1.1 200 OK
Set-Cookie: SonBakilan=3; expires=Tue, 01 Dec 2026 12:18:47 GMT; path=/; samesite=lax; httponly

2) Tarayıcı → sunucu (sonraki HER istekte)
GET /Kitap HTTP/1.1
Cookie: SonBakilan=3
```

İki şeye dikkat edin:

- **Sunucu cookie'yi yazmaz, tarayıcıdan yazmasını ister.** `Set-Cookie` bir talimattır; tarayıcı kabul etmeyebilir (bölüm 2.6).
- **Geri gelen yalnızca `ad=değer` 'dir.** `expires`, `httponly`, `samesite` tarayıcıya verilen talimatlardır ve sunucuya **geri gönderilmez**. `/Durum` sayfasındaki tabloda bu sütunların olmamasının sebebi budur.

2.2. Yazmak: `Response.Cookies.Append`

KampusKitap, bakılan son kitabı hatırlamak için bir cookie yazıyor (**TODO 7.1**, `KitapController.Detay`):

```
Response.Cookies.Append("SonBakilan", kitap.Id.ToString(), new CookieOptions
{
    Expires = DateTimeOffset.Now.AddDays(30),   // yoksa tarayıcı kapanınca silinir
    HttpOnly = true,                            // JavaScript okuyamaz
    SameSite = SameSiteMode.Lax                 // başka sitenin POST'unda gönderilmez
});
```

Ölçülen yanıt başlığı:

```
Set-Cookie: SonBakilan=3; expires=Tue, 01 Dec 2026 12:18:47 GMT; path=/; samesite=lax; httponly
```

`Append`'in yaptığı tek şey yanıta bu satırı eklemektir. `CookieOptions` vermezseniz satır en kısa hâliyle gider: `Set-Cookie: Deneme-Gecici=1; path=/`.

2.3. Okumak: `Request.Cookies["…"]`

Kitap listesi aynı cookie'yi okuyup "son baktığınız kitap" bağlantısını gösteriyor (**TODO 7.2**, `KitapController.Index`):

```
if (int.TryParse(Request.Cookies["SonBakilan"], out var sonId))
    ViewData["SonBakilan"] = KitapDeposu.Bul(sonId);
```

Üç kural bu iki satırda:

1. **Değer her zaman `string` 'dir** — ya da cookie yoksa `null`. Sayıya siz çevirirsiniz.

2. **Değere güvenmeyin.** Cookie istemci verisidir: kullanıcı F12 → *Application* → *Cookies*'ten değeri `abc` ya da `999` yapabilir. Ölçüldü: iki durumda da sayfa hata vermeden açıldı ve bağlantı çizilmedi; `int.Parse` yazılsaydı `abc` sayfayı 500 ile düşürürdü.
3. **Yazdığınız cookie aynı istekte okunamaz.**

Üçüncü kural her yıl şaşırtır. `/Durum/AyniIstek` sayfası aynı action'da önce yazar, sonra okur:

```
Response.Cookies.Append("Deneme-Ayni", DateTime.Now.ToString("HH:mm:ss"));
ViewData["Okunan"] = Request.Cookies["Deneme-Ayni"];
```

İstek	Yazılan	Okunan
İlk açılış	`15:18:45`	**(yok)**
Yenileme	`15:18:46`	`15:18:45`

`Response.Cookies` bu yanıta `Set-Cookie` ekler; `Request.Cookies` ise tarayıcının **bu istekle getirdiklerini** gösterir. Okunan her zaman bir önceki isteğin yazdığıdır.

2.4. `CookieOptions`

Seçenek	Ne yapar	Vermezseniz	Ölçülen
`Expires` (ya da `MaxAge`)	Cookie'nin silineceği an	**Geçici cookie**: tarayıcı kapanınca silinir	F12 → Application'da Expires sütunu: tarih ya da *Session*
`HttpOnly`	JavaScript (`document.cookie`) göremez, yazamaz	JavaScript okur	Bölüm 4.2
`Secure`	Yalnızca HTTPS bağlantısında gönderilir	HTTP'de de gider	Bölüm 4.3
`SameSite`	Başka bir siteden başlayan isteklerde gönderilsin mi?	Satır yazılmaz; karar tarayıcınındır	Bölüm 4.4
`Path`	Hangi adreslerde gönderilir	`/` — bütün site	`path=/` her satırda

Geçici ve kalıcı cookie. `Expires` verilmeyen cookie tarayıcı kapanınca silinir; İngilizce kaynaklarda buna *session cookie* denir. Bu adın ASP.NET Core'un **Session**'ıyla (bölüm 3) ilgisi yoktur — yalnızca tarayıcının açık kaldığı süreyi anlatır. `/Durum/Cerez` sayfasındaki iki düğme farkı gösterir:

Düğme	Yazılan	F12 → Application → Expires
Geçici cookie	`Set-Cookie: Deneme-Gecici=1; path=/`	*Session*
Kalıcı cookie	`Expires = +7 gün`	7 gün sonrasının tarihi

2.5. Silmek: `Response.Cookies.Delete`

Sunucu tarayıcıdaki bir dosyayı silemez; yalnızca **süresi geçmiş** aynı adlı bir cookie yazar, tarayıcı da onu atar:

```
Response.Cookies.Delete("Deneme-Kalici");
```

```
Set-Cookie: Deneme-Kalici=; expires=Thu, 01 Jan 1970 00:00:00 GMT; path=/
```

⚠️ Cookie `Path` ile yazıldıysa silerken aynı `Path` verilmelidir; yoksa tarayıcı onu başka bir cookie sanar ve eskisi durur.

2.6. Sınırlar

Sınır	Ölçülen / kural
Boyut	Tarayıcı ~4096 baytı aşan cookie'yi **saklamaz** ve hata vermez. 4000 karakterlik değer saklandı; 5000 karakterlik değer `Set-Cookie` ile gitti ama tarayıcıda hiç oluşmadı.
Her istekte gider	Cookie resim, CSS, JavaScript isteklerinde bile gönderilir. Büyük cookie her isteği büyütür.
Değer kodlanır	`Append("liste", "12,7,3")` → `Set-Cookie: liste=12%2C7%2C3` . `Request.Cookies` geri çözer: `12,7,3` .
Kullanıcı görür ve değiştirir	F12 → Application → Cookies. Bölüm 4.1.

Bu tablonun sonucu basittir: **cookie'ye küçük ve gizli olmayan şeyler konur** — tema tercihi, dil, son bakılan kitabın numarası. Sepetin tamamı, kullanıcının rolü ya da bir fiyat cookie'ye **konmaz**.

2.7. Sınavın klasik sorusu: cookie okumak

Her yıl bir biçimde sorulur: "*Tarayıcıdan gelen `tema` adlı cookie'nin değeri nasıl okunur?*" Seçeneklerde neredeyse her zaman şu ikisi yan yana durur:

```
var t = Request.Cookies["tema"];    // ✓ istekle GELEN cookie
var t = Response.Cookies["tema"];   // ✗ derlenmez: Response.Cookies'te okuma yoktur
```

Ayrımı yön üzerinden hatırlayın: **istek (request) tarayıcıdan gelir, yanıt (response) tarayıcıya gider.** Gelen cookie'yi `Request` 'ten okursunuz; gidecek cookie'yi `Response` 'a eklersiniz. `Response.Cookies` yalnızca `Append` ve `Delete` bilir. `HttpContext.Session.GetString("tema")` ise başka bir şeydir: session'daki değeri okur, cookie'yi değil.

---

3. Session

Session, veriyi **sunucuda** tutan ve tarayıcıya yalnızca o veriyi bulmaya yarayan bir **anahtar** veren mekanizmadır. Anahtar bir cookie'de taşınır; veri hiçbir zaman tarayıcıya gitmez.

3.1. Fikir: veri sunucuda, anahtar tarayıcıda

```
Tarayıcı                                   Sunucu (bellek)
.AspNetCore.Session=CfDJ8A4%2Bmf96z…  ──►  [anahtar] → { Sepet: [...], Sayac: 2 }
```

Ölçülen ilk yazma yanıtı:

```
Set-Cookie: .AspNetCore.Session=CfDJ8A4%2Bmf96zDRJm2E03vhbV…; path=/; samesite=lax; httponly
```

Çatının yazdığı bu cookie'de üç şey dikkat çeker: **HttpOnly** (JavaScript göremez), **samesite=lax**, ve **Expires yok** — tarayıcı kapanınca anahtar gider. Değer şifrelenmiştir; içinde sepet ya da sayaç değil, sunucudaki kaydın anahtarı vardır.

3.2. Kurmak (1): servisler — TODO 7.3

Session iki adımda kurulur. Birincisi `Program.cs`'in servis kaydı evresidir (Hafta 1), yani `builder.Build()`'dan **önce**:

```
builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(o =>
{
    o.IdleTimeout = TimeSpan.FromMinutes(20);   // 20 dk istek gelmezse silinir (varsayılan)
});
```

Servis kaydı bir şey **çalıştırmaz**; uygulamanın ihtiyaç duyduğunda kullanacağı nesneleri tanıtır (Dependency Injection, Hafta 2). Kayıt uygulama açılırken **bir kez** yapılır. `AddSession` session'ı yöneten servisi (`ISessionStore`) ve ayarlarını kaydeder: `IdleTimeout`, cookie'nin adı (`.AspNetCore.Session`). Bu servis yoksa onu kullanacak middleware de çalışamaz; bölüm 3.3'teki tablonun üçüncü satırı bunu gösteriyor.

`AddDistributedMemoryCache` session verisinin **nerede** duracağını söyler: sunucunun belleği. Bu projede bu satır olmadan da çalıştı — MVC bellek içi önbelleği zaten kaydediyor (ölçüldü). Yine de yazıyoruz: verinin yerini açıkça gösteren ve Redis ya da SQL Server'a geçerken değişecek satır budur (bölüm 3.7).

3.3. Kurmak (2): middleware — TODO 7.4

İkincisi pipeline'a `UseSession` eklemektir:

```
app.UseRouting();
app.UseAuthorization();
app.UseSession();                 // ← session'ı okuyan her şeyden önce
app.MapControllerRoute(...);
```

`UseSession` pipeline'a bir **middleware** ekler. Servislerin aksine **her istekte** çalışır:

1. İstek gelince `.AspNetCore.Session` cookie'sinden anahtarı okur; cookie yoksa yeni bir anahtar hazırlar.

2. O anahtarın kaydını isteğe `HttpContext.Session` olarak ekler. Action ve view session'ı buradan kullanır; veri depodan ilk erişimde yüklenir.
3. Yanıt dönerken session değiştiyse depoya yazar. Session'a ilk kez yazıldıysa yanıta `Set-Cookie` ekler (bölüm 3.4'te ölçüldü).

Sıra kuralı: session'ı **okuyan** her şey `UseSession`'dan **sonra** gelmelidir. Hafta 2'de middleware'lerin sırayla çalıştığını gördük; session, `UseSession` çalışana kadar yoktur. Ölçülen iki durum:

Sıra	Sonuç
Session'ı okuyan kendi middleware'iniz `UseSession`'dan **önce**	**500** — Session has not been configured for this application or request.
`UseSession`, `MapControllerRoute`'tan **sonra** yazılmış	**Çalıştı** — action'lar pipeline'ın en sonunda çalışır

İkinci satır sizi yanıltmasın: kural "`MapControllerRoute`'tan önce" değil, "session'ı okuyandan önce"dir. Microsoft'un önerdiği yer yukarıdaki gibidir — `UseRouting`'den sonra, `MapControllerRoute`'tan önce — ve biz de oraya yazıyoruz.

Eksik kurulumun üç hâli — üçü de ölçüldü, mesajlar birebir:

`AddSession`	`UseSession`	Ne olur
yok	yok	Session'a dokunan sayfa **500**: *Session has not been configured for this application or request.*
var	yok	Aynı **500**
yok	var	Uygulama **açılmaz**: *Unable to resolve service for type 'Microsoft.AspNetCore.Session.ISessionStore' while attempting to activate 'Microsoft.AspNetCore.Session.SessionMiddleware'.*

Üçüncü satır ötekilerden farklıdır: hata bir sayfada değil, **uygulama başlarken** çıkar. Middleware'in ihtiyaç duyduğu servis kayıtlı değildir (Hafta 2'nin Dependency Injection'ı).

> 🍼 Başlangıç projesinde `/Durum/SessionSayaci` sayfasını açın: 500. Yalnızca TODO 7.3'ü yapın: yine 500. TODO 7.4'ü de yapın: sayaç 1.

3.4. Okumak ve yazmak

Session, değerleri bir anahtarla saklar. Çatının verdiği metotlar üç tiple sınırlıdır:

Yazmak	Okumak	Tip
`SetString("Rol", "ogrenci")`	`GetString("Rol")`	`string` — yoksa `null`
`SetInt32("Sayac", 2)`	`GetInt32("Sayac")`	`int?` — yoksa `null`
`Set("Veri", bytes)`	`TryGetValue("Veri", out bytes)`	`byte[]`
`Remove("Rol")` · `Clear()`	—	Bir anahtarı ya da hepsini siler

`/Durum/SessionSayaci` sayfası her açılışta bir sayaç artırır:

```
var sayac = (HttpContext.Session.GetInt32("Sayac") ?? 0) + 1;
HttpContext.Session.SetInt32("Sayac", sayac);
```

Ölçülen davranışlar:

Deneme	Sonuç	Neden
Sayfayı yenilemek	Sayaç artar	Tarayıcı session cookie'sini her istekte getirir
Yeni sekme	Kaldığı yerden artar	Aynı tarayıcı, aynı cookie, aynı session
Gizli pencere	1'den başlar	Gizli pencerenin cookie'leri ayrıdır
Uygulamayı yeniden başlatmak	**1'den başlar**	Veri bellekteydi; cookie duruyor ama işaret ettiği kayıt yok
`IdleTimeout` kadar beklemek	1'den başlar	Hareketsiz session silinir (5 saniyeyle denendi)

Bir ayrıntı daha ölçüldü: **session cookie'si ilk yazmada gelir.** Session'ı yalnızca okuyan bir istek `Set-Cookie` üretmez; ilk `Set…` çağrısının yanıtında gelir. Menüdeki sepet rozeti her sayfada session'ı okuduğu hâlde, sepete bir şey eklemeyen ziyaretçiye session cookie'si verilmez.

3.5. Nesne saklamak: JSON — TODO 7.5 ve 7.6

Sepet bir liste: `List<SepetSatiri>`. Session ise yalnızca `string`, `int` ve `byte[]` bilir; `SetObject` **diye bir metot yoktur.** Çözüm, nesneyi JSON metnine çevirip `string` olarak saklamaktır. Projede bunu iki uzantı metodu yapıyor (**TODO 7.5**, `Services/SessionUzantilari.cs`):

```
public static void NesneYaz<T>(this ISession session, string anahtar, T deger)
{
    session.SetString(anahtar, JsonSerializer.Serialize(deger));
}

public static T? NesneOku<T>(this ISession session, string anahtar)
{
    var json = session.GetString(anahtar);
    return json is null ? default : JsonSerializer.Deserialize<T>(json);
}
```

Sepete ekleyen action (**TODO 7.6**, `SepetController.Ekle`) üç adım yapar — **oku, değiştir, geri yaz**:

```
var sepet = HttpContext.Session.NesneOku<List<SepetSatiri>>("Sepet") ?? [];
var satir = sepet.FirstOrDefault(s => s.KitapId == id);
if (satir is null)
    sepet.Add(new SepetSatiri
    {
        KitapId = kitap.Id, Baslik = kitap.Baslik, Fiyat = kitap.Fiyat, Adet = 1
    });
else
    satir.Adet++;
HttpContext.Session.NesneYaz("Sepet", sepet);
```

> ⚠️ **Geri yazmayı unutmak** sık yapılan hatadır. `NesneOku` JSON metninden **yeni bir liste** üretir; listeye eklemek session'daki metni değiştirmez. `NesneYaz` çağrılmazsa sepet hep boş görünür ve hata da çıkmaz.

`/Durum` sayfası session'daki değeri olduğu gibi gösterir. Bir kitap eklendikten sonra ölçülen:

```
Sepet   [{"KitapId":3,"Baslik":"Veri Yapıları","Fiyat":240,"Adet":1}]
```

Türkçe harflerin `ı` gibi görünmesi bir hata değildir: `JsonSerializer` ASCII dışı karakterleri varsayılan olarak kaçış dizisiyle yazar; geri okurken `Veri Yapıları` olur.

3.6. Her sayfada: View Component rozeti — TODO 7.7

Menüdeki sepet rozeti Hafta 3'teki fikrin tekrarıdır: **View Component veriyi kendi getirir.** Bu kez kaynak bir servis değil, isteğin session'ı:

```
public IViewComponentResult Invoke()
{
    var sepet = HttpContext.Session.NesneOku<List<SepetSatiri>>("Sepet") ?? [];
    return View(sepet.Sum(s => s.Adet));
}
```

Rozet layout'ta durduğu için her sayfada çalışır; hiçbir controller sepet sayısını `ViewBag`'e koymak zorunda kalmaz. Ölçüldü: aynı kitap iki kez eklenince rozet **2** gösterdi.

3.7. Session'ın sınırları

Sınır	Sonucu
Veri sunucunun belleğinde	Uygulama yeniden başlayınca kaybolur (ölçüldü).
Tek sunucu	Uygulama iki sunucuda çalışırsa, birinin belleğindeki session ötekinde yoktur. Çözüm: dağıtık önbellek — Redis ya da SQL Server. `AddDistributedMemoryCache` satırı değişir, kod değişmez (`H07-detay.md` bölüm 3).
Tarayıcıya bağlı	Başka bir cihazdan girildiğinde sepet orada değildir. Kalıcı sepet veritabanına aittir (Hafta 8).
Zaman aşımı	`IdleTimeout` (varsayılan 20 dk) boyunca istek gelmezse silinir.

3.8. Sınavın klasik sorusu: nesneyi session'da saklamak

2021'den beri her sınavda bir biçimde var: "*Session'da bir nesne (ör. bir sınıf örneği) saklamak için tipik olarak ne yapılır?*" Doğru cevap bölüm 3.5'tir: **nesne JSON'a çevrilip `string` olarak saklanır.** Yanlış seçenekler genellikle şunlardır:

Seçenek	Neden yanlış
"`SetObject()` ile doğrudan eklenir"	Çatıda böyle bir metot yoktur; yalnızca `string`, `int`, `byte[]`

"`static` bir değişkende tutulur"	`static` bütün kullanıcılar için **tek** kopyadır; herkes aynı sepeti görür
"`ViewBag`'e konur"	`ViewBag` tek istek yaşar (Hafta 4)
"Cookie'ye yazılır"	Session'ın konusu değil; üstelik veri kullanıcının eline geçer

Açık uçlu biçimi de aynı iki satırdır: yazarken `JsonSerializer.Serialize` + `SetString`, okurken `GetString` + `JsonSerializer.Deserialize<T>`. Okuduğunuz nesneyi değiştirdiyseniz **geri yazmayı** unutmayın.

---

4. Güvenlik: İstemcide Sır Tutulamaz

4.1. Cookie kullanıcının elindedir

Ölçülen: tarayıcıya elle `Rol=yonetici` adında bir cookie eklendi; sonraki istekte sunucunun `Request.Cookies` listesinde **göründü**. Sunucu, bir cookie'yi kendisinin mi yazdığını yoksa kullanıcının mı eklediğini **ayırt edemez** — ikisi de aynı `Cookie` başlığıyla gelir.

Kural: **cookie'deki bir değer, kullanıcının size söylediği bir şeydir; doğru olduğunu gösteren bir kanıt değildir.** Tema tercihi için bu sorun değildir; yetki için ölümcüldür (Klasik Tuzak).

4.2. `HttpOnly`: JavaScript göremez

`/Durum/Cerez` sayfası sunucunun aldığı listeyle `document.cookie`'yi yan yana gösterir. Ölçülen:

Cookie	Sunucu (`Request.Cookies`)	JavaScript (`document.cookie`)
`Deneme-Gecici`	✓	✓
`Deneme-Kalici`	✓	✓
`Deneme-HttpOnly`	✓	—

Ayrıca JavaScript ile `document.cookie = 'Deneme-HttpOnly=hack'` yazmak denendi: değer **değişmedi**. Bu neden önemli? Hafta 3'te XSS'i gördük: saldırganın betiği sizin sayfanızda çalışırsa `document.cookie` ile cookie'leri okuyup kendi sunucusuna gönderebilir. Session cookie'si çalınırsa saldırgan sizin session'ınızla gezer (*session hijacking*). `HttpOnly` bu kapıyı kapatır; bu yüzden çatı session ve anti-forgery cookie'lerini `httponly` yazar.

> `HttpOnly` cookie'yi **kullanıcıdan** gizlemez: F12 → Application'da görünür ve değiştirilebilir. Yalnızca sayfadaki **betiklerden** gizler.

4.3. `Secure`: yalnızca HTTPS

`Secure` işaretli cookie şifresiz HTTP bağlantısında gönderilmez; ağı dinleyen biri onu yakalayamaz. Ölçüm notu: `http://localhost` üzerinde `Secure` cookie **saklandı**, çünkü tarayıcılar `localhost`'u güvenli kabul eder. Gerçek bir sunucuda HTTP üzerinden yazılan `Secure` cookie saklanmaz. Projedeki `UseHttpsRedirection` (Hafta 1) HTTP isteklerini HTTPS'e yönlendirir; `Secure` cookie'ler ancak böyle bir sitede anlamlıdır.

4.4. `SameSite` ve CSRF

Hafta 5'te CSRF'i gördük: başka bir sitedeki form sizin sitenize POST gönderir, tarayıcı da cookie'lerinizi **otomatik** ekler. `SameSite` tarayıcıya bu otomatikliği sınırlamasını söyler:

Değer	Başka siteden başlayan istekte cookie gider mi?
`Strict`	Hayır — bağlantıya tıklayarak gelseniz bile
`Lax`	Yalnızca üst düzey GET gezinmesinde (bağlantıya tıklamak); **POST'ta gitmez**
`None`	Her zaman — `Secure` ile birlikte yazılmak zorunda

ASP.NET Core session cookie'sini `samesite=lax` yazar (ölçüldü); biz de `SonBakilan`'ı öyle yazdık. `SameSite` yazılmazsa satırda bu seçenek hiç olmaz ve karar tarayıcıya kalır; güncel Chrome böyle bir cookie'ye `Lax` gibi davranır.

> `SameSite=Lax` CSRF'e karşı ikinci bir katmandır, anti-forgery token'ın yerini tutmaz. Hafta 5'teki `[ValidateAntiForgeryToken]` yerinde kalır.

4.5. Doğru yer: kimlik ve rol sunucuda

Kullanıcının kim olduğu ve neye yetkisi olduğu **sunucunun** verdiği karardır; sunucuda durur. Bu hafta bunu session ile yapıyoruz (TODO 7.8). Gerçek giriş-çıkış, parolalar ve `[Authorize]` **Hafta 11**'in konusudur; orada kimlik bilgisi yine bir cookie'de taşınır, ama o cookie **şifrelenmiş ve imzalıdır** — kullanıcı değiştirirse sunucu bunu anlar.

---

5. Hangi Veri Nerede?

Veri	Yer	Neden
Tema, dil tercihi	**Cookie** (kalıcı)	Küçük, gizli değil; kullanıcı değiştirse zarar yok
Son bakılan kitabın numarası	**Cookie**	Küçük; okurken doğrulanıyor
Sepet (bu ziyaret)	**Session**	Liste büyüyebilir; fiyat kullanıcının elinde olmamalı
Kullanıcının rolü / kimliği	**Session** (bu hafta), kimlik cookie'si (Hafta 11)	Kullanıcı değiştirememeli
"Kaydedildi" mesajı	**TempData**	Yalnızca bir sonraki isteğe kadar (Hafta 4)
Arama terimi, sayfa numarası	**Query string**	Paylaşılabilir, yer imine eklenebilir
Siparişler, kalıcı sepet	**Veritabanı**	Uygulama yeniden başlasa da, cihaz değişse de durmalı (Hafta 8)

Karar iki soruyla verilir — haftanın ilkesindeki iki soru:

1. **Kime ait?** Kullanıcı değiştirirse zarar var mı? Varsa tarayıcıya konmaz.

2. **Ne kadar yaşamalı?** Bir sonraki istek → `TempData`; bu ziyaret → session; aylarca, her cihazda → veritabanı.

🧭 Sorun Çözme Köşesi

```
Uygulama "hatırlamıyor"
├─ Cookie mi, session mı?
│   ├─ Cookie
│   │   ├─ F12 → Network → yanıtta Set-Cookie var mı? ... yoksa Append çalışmadı
│   │   ├─ F12 → Application → Cookies'te var mı? ......... yoksa tarayıcı reddetti (4 KB? Secure?
│   │   ├─ Sonraki istekte Cookie başlığında var mı? ...... Path / Expires doğru mu?
│   │   └─ Aynı action'da yazıp okuyorsanız ............... okunmaz; bir sonraki istekte gelir
│   └─ Session
│       ├─ 500 "Session has not been configured" ........ AddSession ya da UseSession yok,
│       │                                                  ya da session'ı okuyan şey UseSession'd
│       ├─ Uygulama açılmıyor, ISessionStore ............. UseSession var, AddSession yok
│       ├─ Değer her istekte boş ......................... geri yazmayı unuttunuz (NesneYaz)
│       └─ Yeniden başlatınca kayboldu ................... normal: veri bellekte
└─ "Kullanıcı başkasının yetkisiyle girdi" ............. yetki cookie'de mi? (Klasik Tuzak)
```

Soru	Araç
Sunucu cookie yazmak istedi mi?	F12 → Network → yanıt başlıklarında `Set-Cookie`
Tarayıcı cookie'yi sakladı mı, ne zaman silecek?	F12 → Application → Cookies (Expires, HttpOnly, Secure, SameSite sütunları)
Tarayıcı bu istekte ne getirdi?	F12 → Network → istek başlığında `Cookie` · projede `/Durum`
Session'da ne var?	Projede `/Durum` (session tablosu)

🐛 Hata Köşesi — Hafta 7

Belirti	Gerçek sebep	Çözüm
`InvalidOperationException: Session has not been configured for this application or request.`	`AddSession` ya da `UseSession` yok; ya da session'ı okuyan middleware `UseSession`'dan önce	İkisini de ekleyin; session'ı okuyanı `UseSession`'dan sonraya alın
Uygulama açılmıyor: `Unable to resolve service for type '…ISessionStore'`	`UseSession` var, `AddSession` yok	`builder.Services.AddSession()` — `Build()`'dan önce
Cookie yazdım, `Request.Cookies` boş	Aynı istekte okunuyor	Bir sonraki istekte okuyun; bu istekte değeri zaten biliyorsunuz
Cookie hiç oluşmuyor, hata da yok	Değer ~4 KB'tan büyük	Cookie'ye küçük veri koyun; büyük veri session'a

Sepete ekliyorum, sepet boş	Liste okundu, değiştirildi, **geri yazılmadı**	`NesneYaz` ile geri yazın
`int.Parse(Request.Cookies["…"])` → `FormatException`	Kullanıcı cookie'yi değiştirmiş ya da cookie yok	`int.TryParse`
Uygulama yeniden başlayınca sepet boş	Session verisi bellekte	Normaldir; kalıcı veri veritabanına (Hafta 8)
`document.cookie`'de cookie görünmüyor	Cookie `HttpOnly`	Doğru davranış; JavaScript'in görmesi gerekiyorsa gizli olmamalı

🕳️ Klasik Tuzak — Kullanıcının rolünü cookie'de düz metin tutmak

```
// Girişte
Response.Cookies.Append("Rol", "ogrenci");

// Panelde
var rol = Request.Cookies["Rol"] ?? "misafir";
if (rol == "yonetici") { /* yönetici paneli */ }
```

Kod çalışır, sayfalar doğru açılır, testte her şey yerindedir. Projede `/Durum/Panel` sayfası Başlangıç projesinde tam olarak böyle yazılmıştır. Ölçülen: "Öğrenci olarak giriş yap" düğmesine basıldı (`Set-Cookie: Rol=ogrenci; path=/`), sonra F12 → Application'da değer `yonetici` yapıldı ve sayfa yenilendi: **yönetici paneli açıldı.**

Doğrusu (TODO 7.8): rol sunucuda tutulur.

```
HttpContext.Session.SetString("Rol", "ogrenci");            // girişte
var rol = HttpContext.Session.GetString("Rol") ?? "misafir";  // panelde
```

Bitmiş projede aynı deneme tekrarlandı: elle eklenen `Rol=yonetici` cookie'si sunucuya gitti ama rol **ogrenci** kaldı. Değiştirilecek bir şey yok: rol tarayıcıda değil.

> Aynı hatanın kılık değiştirmiş hâlleri: fiyatı gizli alanda taşımak (Hafta 4), kullanıcı numarasını query string'den almak, "yönetici mi?" bilgisini `localStorage`'da tutmak. Hepsinde veri kullanıcının elindedir.

📖 Terimler Sözlüğü

Terim	Anlamı
State (durum)	İstekler arasında kaybolmaması gereken bilgi
State management	Bu bilgiyi istekler arasında taşımanın yolları

Cookie	Sunucunun tarayıcıya bıraktığı, tarayıcının her istekte geri getirdiği `ad=değer`
`Set-Cookie`	Sunucunun cookie yazdırdığı yanıt başlığı
`Cookie`	Tarayıcının cookie'leri getirdiği istek başlığı
Geçici cookie	`Expires` verilmemiş cookie; tarayıcı kapanınca silinir (İngilizcesi *session cookie*)
Kalıcı cookie	`Expires` ya da `MaxAge` verilmiş cookie
`HttpOnly`	Cookie'yi JavaScript'ten gizleyen seçenek
`Secure`	Cookie'nin yalnızca HTTPS'te gönderilmesi
`SameSite`	Başka siteden başlayan isteklerde cookie'nin gönderilip gönderilmeyeceği
Session	Sunucuda tutulan, tarayıcıya yalnızca anahtarı verilen kullanıcı verisi
`IdleTimeout`	İstek gelmeyen session'ın silineceği süre (varsayılan 20 dk)
Dağıtık önbellek (distributed cache)	Session verisinin birden çok sunucunun ortak eriştiği yerde tutulması (Redis, SQL Server)
Session hijacking	Çalınan session cookie'siyle başkasının session'ını kullanmak

---

✅ Kontrol Soruları

A. Durum

1. HTTP durumsuzken bir sepet uygulaması kullanıcıyı nasıl hatırlar? Üç cümleyle açıklayın.
2. Bölüm 1.2'deki beş yerden hangilerindeki veriyi kullanıcı değiştirebilir?
3. `/Durum` sayfasında gördüğünüz `.AspNetCore.Mvc.CookieTempDataProvider` cookie'sini kim, hangi hafta yazdı?

B. Cookie

4. `Set-Cookie` ve `Cookie` başlıklarından hangisi yanıtta, hangisi istekte bulunur?
5. Sunucu, bir cookie'nin `Expires` ve `HttpOnly` değerlerini sonraki isteklerde öğrenebilir mi? Neden?
6. `Response.Cookies.Append("tema", "koyu");` satırının ürettiği yanıt başlığını yazın.
7. Aynı action'da `Append` ile yazılan cookie `Request.Cookies` ile okunursa ne gelir? Neden?
8. Aşağıdaki kodun iki sorununu yazın:

```
var id = int.Parse(Request.Cookies["SonBakilan"]);
```

9. Geçici cookie ile kalıcı cookie arasındaki fark nedir? Hangi seçenek belirler?
10. Bir cookie nasıl silinir? Sunucu tarayıcıya tam olarak ne gönderir?
11. 5000 karakterlik bir cookie yazdınız; hata almadınız ama cookie oluşmadı. Neden?

C. Session

12. Session verisi nerede durur? Tarayıcıda ne bulunur?

13. `AddSession` ve `UseSession`'dan hangisi `Build()`'dan önce, hangisi sonra yazılır?
14. Yalnızca `UseSession` yazılıp `AddSession` unutulursa ne olur? Yalnızca `AddSession` yazılırsa?
15. `UseSession`'ın yeri için kural nedir? "`MapControllerRoute`'tan önce" demek neden eksiktir?
16. Session'a bir `List<SepetSatiri>` nasıl yazılır? Kodu yazın.
17. Bir öğrencinin sepete ekleme kodu çalışıyor ama sepet hep boş görünüyor. En olası sebep nedir?
18. Uygulama yeniden başlatıldığında session'daki sepet neden kaybolur? Tarayıcıdaki cookie de kaybolur mu?
19. Session'ı yalnızca okuyan bir istek `.AspNetCore.Session` cookie'si üretir mi?
20. Uygulama iki sunucuda çalışıyorsa bellek içi session neden sorun çıkarır? Çözüm nedir?

D. Güvenlik

21. `HttpOnly` neyi engeller, neyi engellemez?
22. `SameSite=Lax` yazılmış bir cookie, başka bir sitedeki formun POST isteğinde gönderilir mi?
23. Kullanıcının rolünü cookie'de tutmanın açığını adım adım gösterin. Doğru yer neresidir?

E. Karar

24. Şu verilerin her biri için yer seçin ve gerekçelendirin: tema tercihi · sepet · "silindi" mesajı · sipariş geçmişi · arama terimi.
25. "Fiyatı sepete eklerken cookie'ye yazalım, ödeme sayfasında oradan okuyalım." Bu öneriyi değerlendirin.

📑 Kaynakça

> Bağlantıların sonundaki `?view=aspnetcore-10.0` **kasıtlıdır**: sürüm sabitlenmezse arama motoru sizi eski sürüme düşürür.

Kaynak	Dayandırdığı bölüm
Session and state management in ASP.NET Core — https://learn.microsoft.com/aspnet/core/fundamentals/app-state?view=aspnetcore-10.0	Bölüm 1, 3 — state seçenekleri, session kurulumu, `Set/Get` metotları, JSON ile nesne
CookieOptions Class — https://learn.microsoft.com/dotnet/api/microsoft.aspnetcore.http.cookieoptions?view=aspnetcore-10.0	Bölüm 2.4 — seçenekler
Work with SameSite cookies in ASP.NET Core — https://learn.microsoft.com/aspnet/core/security/samesite?view=aspnetcore-10.0	Bölüm 4.4
MDN — Using HTTP cookies — https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies	Bölüm 2.1, 2.6, 4 — başlıklar, sınırlar, `HttpOnly`, `Secure`, `SameSite`
OWASP Session Management Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html	Bölüm 4 — session hijacking, cookie seçenekleri

📚 Daha Fazlası

`H07-detay.md` dosyasında (**sınav kapsamı dışı**):

- Cookie'yi imzalamak ve şifrelemek: Data Protection (bölüm 1)
- `TempData`'yı cookie yerine session'da tutmak (bölüm 2)
- Dağıtık session: Redis ve SQL Server (bölüm 3)
- Session'ın içi: kilitlenmez, `LoadAsync`, istek sonunda yazma (bölüm 4)
- `SameSite`'ı iki siteyle ölçmek — ve `SameSite` yazılmayan cookie'nin iki dakikası (bölüm 5)
- Cookie önekleri: `__Host-` ve `__Secure-` (bölüm 6)
- `localStorage` ve `sessionStorage`: tarayıcıda durumun öteki iki yeri (bölüm 7)

🔜 Haftaya Hazırlık

Bu hafta sepet session'da duruyordu ve uygulama yeniden başlayınca **kayboldu**. Kitap listesi ise baştan beri bellekteki bir listeydi: eklediğiniz kitaplar da yeniden başlatınca gidiyordu.

Hafta 8'de veriyi kalıcı hâle getireceğiz: **Entity Framework Core** ve **Code First** ile bir veritabanı.

Düşünerek gelin: `KitapDeposu` şu an bir `static List<Kitap>`. Uygulama iki ayrı bilgisayarda çalışsaydı, birinde eklenen kitap ötekinde görünür müydü? Bu hafta session için sorduğumuz soru — *veri nerede duruyor?* — kitap listesi için ne söylüyor?

> 🏠 Ev göreviniz bu haftanın sınav provasıdır: kendi projenizde bir **favori listesi** ya da **sepeti** session'da, nesneyi JSON'a çevirerek tutun.